

A PROJECT REPORT
ON
“Autonomous AI in Navigation and Visualization”

A project report submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology in Artificial Intelligence & Machine Learning

**BACHELOR OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE &
MACHINE LEARNING**

Submitted By

Sanskar Jadhav
PRN 21070126076

UNDER THE GUIDANCE OF

Dr. Kalyani Kadam

Assistant Professor



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Pune – 412115, Maharashtra State, India

<https://www.sitpune.edu.in/>

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

AY 2024-25



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Pune – 412115, Maharashtra State, India

<https://www.sitpune.edu.in/>

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that the Project work entitled “**Autonomous AI in Navigation and Visualization**” is carried out by **Sanskar Jadhav**, in partial fulfillment for the award of the degree of **Bachelor of Technology in Artificial Intelligence & Machine Learning**, Symbiosis International (Deemed University), Pune during the academic year 2024-2025.

Mr. Deepak Jaiswal
Industry Mentor

Dr. Kalyani Kadam
Guide

Dr. Shruti Patil
Head, Department of AI&ML

DECLARATION

I hereby declare that the project titled “**Autonomous AI in Navigation and Visualization**” submitted to Symbiosis Institute of Technology, Constituent of Symbiosis International (Deemed University) Pune for the award of the degree of Bachelor of Technology in Artificial Intelligence & Machine Learning is a result of original research carried out by me. It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of any degree or diploma.

Name(s) of Student(s): Sanskar Jadhav

PRN: 21070126076

Degree: Bachelor of Technology in AI&ML

Department: Artificial Intelligence & Machine Learning

Title of the project : Autonomous AI in Navigation and Visualization

(Signatures of the Students)

Date: 13/11/2024

ACKNOWLEDGEMENT

I want to express my sincere gratitude to everyone who supported me throughout this project. First and foremost, I would like to thank my project guide, **Dr. Kalyani Kadam**, for her valuable guidance, encouragement, and feedback. She has been a constant source of inspiration and motivation for me.

I would also like to thank the head of the department, **Dr. Shruti Patil**, for providing me with the necessary facilities and resources for conducting this project. I am grateful to her for providing constant support and advice.

I want to acknowledge my industry mentor, **Mr. Deepak Jaiswal**, Head of New Technology Exploration at Kalyani Group, for his guidance in every project stage. He has been extremely supportive throughout this journey.

I would also like to thank my family and friends for their love, care, and support. They have always been there for me in times of need and stress. They have encouraged me to pursue my passion and achieve my goals.

Lastly, I thank **Symbiosis Institute of Technology Pune** for allowing me to work on this project and enhance my skills and knowledge. I am proud to be a part of this prestigious institution.

ABSTRACT

Autonomous navigation and visualization are increasingly significant topics of recent interest and remain as the focus of intensive research in allowing AI robots to navigate and identify objects in their surroundings. Deep reinforcement learning (DRL) is instrumental for autonomous exploration that works well with optimizing decision-making under uncertainty, allowing agents to explore unmapped environments in the absence of human control. For custom object detection, YOLO continues to excel in terms of accuracy and latency, with the new Segment Anything Model 2 from Meta providing the possibility for quick annotations of multiple objects recorded in a video. This project entails hyperparameter tuning and training of the TD3 algorithm for indoor navigation in a 30x30m closed environment representing an office workspace, development of a GUI for local usage of SAM 2 to maintain data privacy, and custom training of YOLOv9 for real-time object detection to be put onboard a quadruped robot for real world testing. Experimental results show that the DRL system achieves close results to the ones of a path planner in an already known environment, thereby eliminating the need for storing a map, SAM 2 can be modified to accelerate dataset creation for YOLO training on multiple classes, and with CUDA support, inference time is minimized when using heavier YOLO models for real-time applications in robotics. These results are part of the ongoing development of my work on embedded autonomous AI for Unmanned Ground Vehicles (UGVs).

Keywords: Autonomous navigation, Deep reinforcement learning, TD3, Object detection, YOLO, SAM 2, UGV

TABLE OF CONTENTS

	Page
Certificate	i
Declaration	ii
Acknowledgment	iii
Abstract	iv
Table of Contents	v
List of Tables	vi
List of Figures	vii
List of Abbreviations	viii
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem statement	2
1.3 Scope of research	3
1.4 Research hypothesis	4
1.5 Objectives	4
1.6 Organization of the report	5
CHAPTER 2: LITERATURE REVIEW	6
2.1 Background	6
2.2 Summary of literature review and research gap	17
CHAPTER 3: SOFTWARE AND HARDWARE REQUIREMENTS SPECIFICATION	19
3.1 Software tools	19
3.2 Hardware tools	21
3.3 Work breakdown structure	22
3.4 Functional requirements	24
3.5 Non-functional requirements	25
CHAPTER 4: METHODOLOGY	27
4.1 Autonomous navigation	27
4.2 Autonomous annotation	39
4.3 Autonomous visualization	42
CHAPTER 5: RESULTS AND DISCUSSION	44
CHAPTER 6: CONCLUSION AND FUTURE SCOPE	49
5.1 Conclusion	49
5.2 Future Scope	49
REFERENCES	51
APPENDICES	
AIC Form	
Similarity Report	
AI Plagiarism Report	
Certificate from Industry	

LIST OF TABLES

		Page
Table 1	Evolutions in DRL algorithms and applications in research	10
Table 2	Recent trends in object detection models and applications in research	13
Table 3	Software tools used in the project	19
Table 4	Hardware tools used in the project	21

LIST OF FIGURES

		Page
Figure 1	DRL simulation in 5x5m maze-like environment	31
Figure 2	DRL simulation in a 30x30m workspace environment created from a Point Cloud map	32
Figure 3	Web application for autonomous annotation using Voila in Jupyter Lab	39
Figure 4	Flowchart of sequence of operations in the annotation application	40
Figure 5	Comparison of standard and optimized TD3 algorithm performance	44
Figure 6	Example of negative segmentation masks in spectacles annotation	45
Figure 7	Precision-recall curve for YOLOv8s on dataset of 300 images	46
Figure 8	Precision-recall curve for YOLOv9m on dataset of over 10000 images	47
Figure 9	Normalized confusion matrix for YOLOv9m	47
Figure 10	Graphs for loss, precision, and recall for training and validation across 50 epochs for YOLOv9m	48

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AGI	Artificial General Intelligence
CARML	Collision Avoidance Robotics with Meta-Learning
CNN	Convolutional Neural Network
CT	Computed Tomography
CUDA	Compute Unified Device Architecture
DDPG	Deep Deterministic Policy Gradient
DNN	Deep Neural Network
DQN	Deep Q Network
DDQN	Double Deep Q-Network
DRL	Deep Reinforcement Learning
GPU	Graphics Processing Unit
GUI	Graphical User Interface
LADet	Light-weight and Adaptive Network
LiDAR	Light Detection And Ranging
LSTG	Latent Space Token Generation
MRI	Magnetic Resonance Imaging
MS COCO	Microsoft Common Objects in Context
OU	Ornstein-Uhlenbeck
PPO	Proximal Policy Optimization
PRM	Problem Road Map
R-CNN	Region-based Convolutional Neural Network
ReLU	Rectified Linear Unit
RGB	Red Green Blue
RL	Reinforcement Learning
ROS	Robot Operating System
RRT	Rapidly-exploring Random Tree
SAM	Segment Anything Model
SA-V	Segment Anything Video
SLAM	Simultaneous Localization And Mapping
SSD	Single Shot Detector
TD3	Twin Delayed Deep Deterministic Policy Gradient
UGV	Unmanned Ground Vehicle
ViT	Vision Transformer
VOC	Visual Object Classes
YAML	YAML Ain't Markup Language
YOLO	You Only Look Once

Chapter 1

Introduction

1.1 Introduction

UGV navigation involves multiple dimensions of technologies that cover the control of the vehicles themselves, the route planning, and communications with other services. Each aspect has been positively influenced by recent developments in the field of artificial intelligence and robotics. Of all these technologies, Deep Reinforcement Learning (DRL) seems to be the most effective when optimizing decision making in unpredictable and unstable environments. Within DRL, systems enable exploration of unknown territories within which all navigation is performed autonomously and learned actions are adjusted with experience. Thus, it fits perfectly for situations which do not allow traditional mapping and planning approaches.

A critical element of autonomous navigation is environmental perception, where the UGV is required to recognize, locate and react to the presence of many different objects. Object detection is usually performed using complex CNN based machine learning models; the You Only Look Once (YOLO) framework continues to be the most prominent due to its optimized trade-off between speed and accuracy. This ability, coupled with Meta's Segment Anything Model 2 (SAM 2) creates a highly effective strategy for real-time object detection, allowing quick instance segmentation of selected objects for easier annotation of datasets. These models as a whole improve the performance of the UGV in terms of accurate custom object recognition, which is especially challenging in crowded small indoor areas like offices.

The objective of this study is to develop and optimize an autonomous navigation system for the quadruped robot working within an office-like environment, specifically a 30x30m closed space. The guidance system is based on the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm for DRL-based navigation and integrated with YOLOv9 for real-time object detection. It intends to attain levels of performance on par with traditional path-planning methods, without pre-mapped environments. This includes the development of a local GUI for SAM 2 to ensure data sovereignty and efficient operation through rapid, precise autonomous annotation without reliance on continuous manual intervention. The navigation performance, object detection performance, and overall performance of the developed system will be validated using real-world trials with the robot, within the confines of a closed environment, with contributions to the vast field of embedded autonomous AI applicable for UGVs.

1.2 Problem Statement

While autonomous navigation and object detection technology is progressing quickly, few substantial challenges remain to be solved in the deployment of appropriate systems that can operate in real-time and reliably within confined indoor spaces. While the path-planning traditional methods that exist yield reasonable performance in controlled environments, they highly depend on pre-mapped data, which restricts its adaptability against unknown environments as those natural environments require extensive storage provision and mapping. Thus, the challenge is to navigate autonomously and dynamically with real-time efficiency within the framework of the unmapped environment, with specific attention on an indoor workspace, where accuracy and obstacle avoidance are essentially critical, and is often characterized with several cluttered static and dynamic obstacles.

Moving a step further, even though real-time detection models such as YOLO are advanced, the processes of training and annotating the dataset have proven to be highly resource-intensive and voluminous, most especially in situations where the parameters of data collections require that local processing be done due to privacy constraints. Thus, there lies a dual prospect: efficient autonomous navigation, which could work without a map; secondly, develop a local hosted data annotation application that feeds data for training high-accuracy object detection models in situ.

1.3 Scope of Research

This study aims to design an end-to-end solution for autonomous navigation and object detection in a chosen constrained indoor space – an office environment with 30x30m dimensions. The research will be focused on three main approaches as described below.

Navigation and Exploration via DRL: The focus of this research area will be on improving the TD3 algorithm for Indoor Workspace Map-less Navigation specifically designed for the quadruped robot. The study will compare the effectiveness of navigation using a DRL approach relative to a conventional path-planning approach.

Object Detection and Annotation: Training of the YOLOv9m model with emphasis on real-time accurate detection using specially created custom datasets unique to the workspace setting. In a bid to ensure the safeguarding of data, the system employs SAM 2, which is used locally as an annotating platform for expediting the process of developing multiple object class training datasets.

Evaluation of System Performance in Real World Settings: The system is deployed on the robot using an Nvidia Jetson Orin board with the aim of understanding how well it achieves

the desired level of navigation accuracy, the latency of object detection, and general indoor scenario reliability in UGV navigation.

Thus, the scope of the research implies a long-standing commitment to algorithmic modeling, systems integration, and practical application in the real world with the intent of providing a useful and workable system for autonomous navigation in ground vehicles that will also have an application in object based assisted navigation systems.

1.4 Research Hypothesis

The central hypothesis of the research work is that optimizing the TD3 DRL algorithm for indoor environments along with the integration of an efficient and locally implemented object detection model would result in autonomous navigation and real-time object recognition in a UGV, analogous to that done by conventional path planners for known environments. The hypothesis is based on the idea that strict tuning of hyperparameters for the TD3 algorithm would enable a UGV to navigate without an existing map, while YOLOv9, based upon local annotations provided by SAM 2, can be capable of maintaining accuracy and low latency in object detection.

1.5 Objectives

- 1) Development and rigorous tuning of the TD3 algorithm in such a way as to make autonomous navigation possible in an office workspace-sized 30x30 m area without pre-existing maps or external localization aids.
- 2) Developing an in-house GUI for an autonomous annotation application using a modified version of Meta's SAM 2 model to provide rapid annotations for any custom object classes that might be relevant in the target environment using videos as input.

- 3) Train YOLOv9 for real-time object detection with annotated datasets created with SAM 2 so that office navigation can cooccur with accurate real-time object detection without producing a latency that is too high even with larger model configurations.
- 4) Evaluate the performance of the TD3 navigation in simulated environments by considering travel time, success to collision ratio, and robustness to changing environments.
- 5) Test the algorithms on a real quadruped robot and obtain feedback about the system's navigation accuracy, response latencies, and consistency in object detection in more realistic spaces of interest.

These objectives jointly strive to develop a solution for scalable autonomous navigation and obstacle detection, which improves the current capabilities of UGVs in dynamic cluttered indoor environments, thus providing value for both academic research and industrial applications in autonomous robotics.

1.6 Organization of the Report

The project report continues in the following manner. Chapter 2 includes an in-depth survey of existing literature in the chosen fields of research, with an analysis of the evolution of RL algorithms being used for navigation and CNN architectures for object detection. Chapter 3 discuss the software and hardware requirements for the research. The methodology and technical approach is covered in Chapter 4, with an explanation for the specific models chosen and highlighting the overall architecture of the proposed system. Chapter 5 summarizes the obtained results from both simulated experiments and real world testing with metric logs and graph visualizations. Chapter 6 concludes this report with a synopsis of the research carried out in this project and a discussion on possible future scopes, applications in various industries, and challenges that are yet to be overcome.

Chapter 2

Literature Review

2.1 Background

There has been a gradual growth from traditional static path planning algorithms to deep learning and reinforcement learning approaches. Over the years, a lot of emphasis has been put in the resilient coordination of multi robot systems using deep reinforcement learning. The operation of robots performing real complex tasks like avoiding and dealing with dynamic obstacles, planning paths in uneven terrains and even navigating indoors and outdoors with the presence of other humans and or robots has been achieved to a certain extent. Classical motion-planning approaches are useful for application purposes in a few ordered systems but cannot easily be modified for use in real environments that are characterized by uncertainty and chaos. For this reason, there have been increasing cases of a hybrid use of DRL alongside classical algorithms as a possible solution. These developments have influenced the research of autonomous navigation to progress towards continuous adaptation, policy transfer, and learning in different environments.

The application of open source frameworks have greatly enabled the implementation of DRL for autonomous navigation purposes. The Robot Operating System [1] tackled the various aspects of robotic communication, control protocols as well as inter-sensor integration which led to the fast experimentation and operationalization of deep reinforcement learning solutions. In addition, Gym by OpenAI authored by Brockman et al. [2] provided a unique and adaptable simulation system, which provided an opportunity for researchers to create and update reinforcement system learning policies under reliability before getting to the actual robots. As a result, these frameworks have made it

possible to create a rapid prototyping cycle for many experiments with new navigation algorithms in robotics especially deep reinforcement learning.

The first attempts of using DRL for autonomous navigation started with the elementary navigation and exploration problem in learning based approaches. In [3], Tai and Liu introduced deep reinforcement learning as a technique to enhance cognitive exploration of mobile robots supported by Q-learning networks for indecisive exploration. This approach explained the need for learning in unfamiliar surroundings and dismissed model-based planning to this introduction of decision-making process for mobile robots. In their next advancement [4], they took these principles a step further by adding Q-learning within a frame that enabled robots to tackle more intricate challenges and, without the need for explicit map construction, choose navigational paths, thereby reinforcing the integration of DRL in exploration of mobile robots.

[5] advanced the field by creating a new approach of learning based on simulating the processes of operating mobile robots and transferring it into the real world. This approach covered the process of first training policies in virtual environments and then exporting the learning to real life. The virtual-to-real paradigm allowed the mobile robots to perform reasonably well even if the learning did not involve map based navigation or board level design in path activation which is the typical factor that brings down models of deep reinforcement learning trained only using simulated environments. The turning point here is the fact that policy transfer was made possible and this played a key role in enhancing the use of DRL navigation models in practice.

Simultaneously, [6] improved the autonomous ground robot's learning process, known as end-to-end learning, by integrating a motion planning method capable of making navigation decisions from sensory perception data in a purely data-driven manner. This

implementation of the end-to-end navigation model reduced the redundancies found in the processes by which data is interfaces making it possible to render navigation decisions based purely on data from the given sensors. This organized the navigation processes effectively in order of rank as well as a hierarchy, which was an advantage for this system in fast paced environments, for instance, cities which are very busy or active environments.

The integration of DRL and classic planning strategies represented the next development stage and was influenced by the need for efficient long-range navigation systems that are scalable and can be modified. In [7], the PRM-RL framework was claimed where the Problem Road Map (PRM) was incorporated into reinforcement learning to solve prolonged large-scale navigation challenges. The hybrid design of PRM-RL advanced long navigation by providing a proper ratio between exploration and exploitation, and thus producing a better and more efficient model in regard to the long-range path planning in the complex scenario. Almost in the same vein, [8] presented the modified configuration for PRM-RL with focus on indoor navigation, which increased the usage of the system in constrained spaces, with plenty of obstacles within, like stores or houses, also, allowing the design to extend the dispersal and precision of the PRM-RL in-depth indoor environments.

The hybrid algorithm called RL-RRT [9] integrates Rapidly-exploring Random Tree (RRT) with reinforcement learning to allow robots to execute kinodynamic motion planning using learned reachability estimators. In this scheme, the RRT structure was enhanced by applying RL policies such that the famous tree expansion algorithm could dynamically change according to the learned navigation strategy, one of the best adaptive strategies useful for real-time navigation in unpredictable and dynamic environments.

As navigation techniques underwent improvisations, researchers opted to use more elaborate techniques such as collision avoidance approaches and adaptive control techniques. [10] considered a new type of navigation approach that consisted of a path-planner and an adaptive motion control system, which enables the robot's geometry modification depending on external conditions in real-time. This adaptive model was useful in avoiding dynamic obstacles, as it allowed the robots to adjust their trajectory while avoiding collisions in real time, thanks to captured images from the surrounding environment. In parallel, Iyer and Mahadevan [11] developed Collision Avoidance Robotics with Meta-Learning (CARML), a meta-learning architecture specifically directed at the challenge of dynamic space collisions. CARML permitted robots to learn from past occurrences, so their ability to adapt when faced with new hostile barriers was enhanced.

For tackling training inefficiency, [12] introduced Reverse Curriculum Generation, wherein trainee robot agents are given tasks of progressively increasing difficulty. This so-called 'curriculum learning' enabled the robot to progress through the execution of various tasks in an orderly way and as a result, enhanced learning convergence of robots over different navigation tasks. In the same case, [13] worked on accelerated methods in DRL and explained enhancing control while decreasing the training timeframe which is increasing the flexibility, performance and time efficiency, especially posing a challenge for robotics in field applications.

In the case of actor-critic techniques in DRL, errors due to function approximation often lead to the instability of the policy which is difficult in terms of navigation reliability. To address this concern, [14] implemented the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm, an extension of the existing deep reinforcement learning

approaches with the reduction of the function approximation error. TD3 became a game-changer with respect to practical applications of reinforcement learning for robots seeking optimal policies that were robust and effective during navigation.

Table 1 covers a summary of recent evolutions in DRL approaches.

Table 1. Evolutions in DRL algorithms and applications in research

Ref	Algorithm	Application	Key Findings
[15]	Mapless local path planning using DRL	Dynamic obstacle avoidance	Real-time navigation proves effective however there are issues with stability in areas with a lot of obstacles
[16]	DQN based transition model embedded reinforcement learning	Multi-agent navigation with awareness of social contexts	Self-supervised pretraining participants learn to behave socially appropriately, but it is time-consuming due to the modelling predictive coding and sequence issues involved
[17]	Active SLAM with DRL	Autonomous path planning	Exploitation is combined with SLAM and DRL; however, this approach is highly demanding in terms of processing power
[18]	A LiDAR and DRL based path planner for mobile robots	Indoor mobile robot navigation	Given high path fidelity for known environments, the limitation arises due to lack of coverage expansion into unexplored areas

[19]	Goal oriented DRL and path planning aided obstacle avoidance	Continuous obstacle avoidance	Supports smooth actions in a continuous space with practice, but limited in that implementation without practice leads to overfitting in simulations
[20]	DDPG for continuous control	Autonomous driving	Employs action control in continuous spaces; controls are difficult to tune and not robust in variable environments
[21]	Proximal Policy Optimization (PPO)	High-speed autonomous navigation	Works great where the world is dynamic and continuous, though it is hyperparameter sensitive for reaching stability
[22]	Dueling DDQN for complex navigation	Multi-room pathfinding	Improvement by value function approximation, expending computational resources on two concurrent networks
[23]	Learning based control for an end-to-end deep reinforcement navigation system	Mobile robot navigation	Simplifies training for solutions providing direct navigation given usual conditions of operation; however the system is sensitive to changes in environment
[24]	Target-driven visual navigation using DRL	Indoor exploration	Centralized approach works very efficiently within corridors but does not transfer well to novel configurations

[25]	DRL for map-based learning	Autonomous map navigation	Indoor static maps work well; however high frequency dynamic changes adaptation is limited
------	----------------------------	---------------------------	--

Improvements in object detection throughout the past decade can be largely attributed to the rise of neural network model implementations and the increased access to reliable data for anyone to perform custom training. In the past, models such as Faster R-CNN added region proposal networks which made it possible to accurately detect objects in motion. However, attention has predominantly moved towards networks such as YOLO and SSD built for real time performance due to the lightweight needs of the applications.

Pascal VOC and MS COCO were some of the earliest databases and performed general evaluation of object detection with more recent examples like Cityscapes and various in-house datasets being developed for rescue missions and other applications like looking for a person in a crowd, self-driving in cars, and house surveillance and security. Even latest applications illustrate this versatility of models from real time license plate identification to analyzing detailed space images from satellites.

This growth in technology is directed towards models capable of maintaining a certain level of performance index while being computationally inexpensive for satisfying low latency requirements of many applications. The overall trend can be observed in Table 2, which notes the some of the object detection models being used in research with their applications.

Table 2. Recent trends in object detection models and applications in research

Ref	Year	Model	Application	Key Findings
[26]	2022	Mask R-CNN	Pedestrian detection	It copes with scale and illumination and rotation changes, though it is resource intensive
[27]	2022	Extension of Haar Cascade algorithm	Face recognition for law enforcement and security	It is applicable in real-time settings, though its performance is restricted by light and surrounding
[28]	2021	Weakly supervised DNN	Pedestrian detection	Employs weak segmentation to enhance detection, though does not do well with objects that are highly occluded
[29]	2021	YOLO	License plate detection	Accuracy lowers when given varying determinants of lighting conditions
[30]	2020	SSD-based approach	Vehicle and wheel detection	Vehicle detection system is robust, though inaccurate for varying types of wheels
[31]	2020	Parallel down-up fusion network	Salient object detection in remote images	High precision met with equally high computational cost

[32]	2019	Custom lightweight network	High-speed object detection	Up to 200 FPS is achieved, at the cost of some accuracy
[33]	2019	LADet	Multi-scale object detection	Light and adaptive, but does not cope well with miniscule objects
[34]	2018	Random Access Memories	Target detection in remote sensing	New approach that addresses issues with high-resolution imageries, highly computational
[35]	2018	Spatiotemporal prior with geodesic distance	Video object segmentation	Geodesic distance is good for tracking but not in the presence of crowd scenes
[36]	2017	CNN-based approach	Traffic light detection and tracking	These two tasks can be performed efficiently, but likely that new patterns will need retraining
[37]	2017	Faster R-CNN	General object detection	Speed and accuracy trade-off is optimized for larger objects, struggles with smaller ones

Unveiled by the research facility Meta AI under Meta in 2023, the Segment Anything Model (SAM) has transformed the approach of image segmentation by incorporating segmentation for any target image without the need for training and instead on a prompting basis, making it geared for more computer vision tasks that were limited in terms of dataset availability. The core design of the foundational model SAM uses is a transformer-based architecture and has a prompt-engineered approach, which causes segmentation tasks to

be performed guided by inserted elements such as points, bounding boxes or freehand masks. Since its launch, SAM has shown a notable degree of applicability and versatility in various fields such as but not limited to, pedestrian and road signs detection for cars, healthcare diagnostics specifically medical imaging, and virtual reality. However, the initial approach had challenges in dealing with data types other than images and in multiple object segmentation with tracking, hence prompting more modifications and the subsequent release of SAM 2.

The above challenges were tackled by SAM 2 with improvements such as including video segmentation, inclusion of memory attention for better temporal coherence and also introduction of a broader dataset, SA-V. Consequently, SAM 2 provides enhanced capabilities for tasks that involve constant monitoring of moving objects within the same scene, which includes autonomous vehicles and analysis of medical videos.

Kirillov et al. in [38] introduced SAM emphasizing on instance segmentation for generic purpose using a ViT backbone and training on vast datasets. A commendable advantage of SAM is also the fact it can be used for zero-shot segmentation without adapting it to the target domain; nevertheless, the model is less effective in segmented objects situated in very close proximity and especially where the object encroaches in another (overlapping bounding boxes).

Zhang et al. in [39] performed an early survey on SAM, analyzing its impact on various vision tasks. They found SAM to be advantageous in static image segmentation but not very effective where a fine segmentation of a small or highly occluded target object is required especially when there are few prompts.

Zhu et al. in [40] looked into the many uses of SAM and its application to medical imaging in particular. They found SAM to be especially useful in the process of segmenting the

organs and the lesions present in them. Still, a few restrictions were highlighted in complicated medical scans as SAM's architecture, being of general purpose, was sometimes too broad and precision dropped when considering specific anatomical details.

In [41], Ma et al. assessed the effectiveness of SAM in segmentation of medical images across male and female datasets and findings showed that, although SAM sufficed in MRI segmentation, performance levels dropped in other imaging modalities such as lower contrast anatomy like ultrasound and CT scan. Here, assessment of SAM indicates handling of medical imaging is flawed despite its innovations.

Li et al. in [42] assessed performance of segmentation assisted by multi-modality data using Fusion SAM for imparting interplay of vision and non-vision sensors to incorporate LSTG and multidimensional imaging radar in one system. Clear benefits in segmentation accuracy were noted in detection-integrated driving applications; the model, however, still needs a lot of resources which hinders its expansion.

In [43], Ravi et al. introduced a new version for the segmentation of videos called SAM 2 and highlighted some memory attention mechanisms that achieve temporal coherency of segmentation outputs over time. This component was helpful in scenarios where tracking multiple moving objects for extended periods is essential but again handled video data would need more memory than what most devices can accommodate.

Zhang et al. in [44] evaluated how well SAM 2 performs in video segmentation tasks, especially on how the system enhanced frame to frame segmentation. The memory bank configuration helps to improve the performance of SAM 2 in terms of scene adaptability as compared to the former; nevertheless, challenges concerning the speed of processing and storage effectiveness over extended length videos remain.

Zhang and Shen in [45] analyzed SAM 2's advancements in biomedical segmentation, particularly in dense tissue segmentation for surgical videos. The memory-guided attention provides stable segmentation across sequences, though precision remains an issue in low-contrast regions, where over-segmentation may occur.

Yu et al. in [46] examined SAM 2's deployment in robotic perception tasks, specifically aimed at surgery, utilizing the model's temporal memory to improve consistency in object segmentation. SAM 2 demonstrated superior resilience in real environments, though it has limitations in processing speed and recognizing smaller vessels when onboard precise surgical robotic arms.

2.2 Summary of Literature Review and Research Gap

The evolution of autonomous navigation systems has transcended the conventional static path planning methods to dynamic deep reinforcement learning (DRL) algorithms, which allows robots to execute challenging missions such as dynamic obstacle avoidance, path planning on rough terrain, and navigation through a continuously changing environment. DRL has successfully been applied for multi-robot systems without restricting motion and over a variety of continuous learning environments. With ROS and OpenAI's Gym availability, motion planning works can be enhanced dramatically as these libraries provide adjustable simulation environments for fast and effective implementation of DRL.

Alongside, speed-oriented models like Faster R-CNN, YOLO, and SSD are gaining more prominence for object detection. These models have been commonly used in activities from monitoring traffic and developing driverless cars to building custom systems trained on Pascal VOC, MS COCO, Cityscapes dataset, etc. However, dealing with real-time massive high motion environments has its challenges as far as deployment of the above models for practical applications is concerned.

Meta has made significant strides in segmentation tasks with Segment Anything Model (SAM) and its follow-up remodeling of the same, SAM 2, that allow for zero-shot segmentation across varying domains. Still when it comes to video segmentation of objects in frames in real time and in particular for objects located in the same region, SAM technology proves inadequate. Even SAM 2, which presents video segmentation and memory attention for better tracking with time coherence, suffers in memory usage limiting onboard deployment.

Although DRL, object detection, and SAM have come a long way on their own, there is still an inadequate integration of such systems into one framework. The use of real-time object detection together with segmentation using SAM and navigation models based on DRL has not been fully investigated. The cost of computation and resource consumption required for this task represents a challenge that impedes these systems' adoption in real world scenarios, especially when the resource availability is limited and the systems operate in a dynamic environment. An important aspect of the future development of these technologies is their integration into a single flexible system for object detection, segmentation and real time response management in different environments.

Chapter 3

Software and Hardware Requirements Specification

3.1 Software Tools

Table 3. Software tools used in the project

Software Tool	Purpose	Description
ROS 2 Foxy (Ubuntu 20) & ROS 2 Humble (Ubuntu 22)	Communication, control, and integration for the complete autonomous system	ROS 2 is used as the main platform and provides real-time communication of sensors, controllers, and ML algorithms. The modularity of ROS 2 and support for distributed systems made it very suitable for complex navigation control management.
Gazebo	Simulation environment for testing and optimization of autonomous navigation	Allows creation of 3D simulation spaces where the quadruped robot can interact with virtual obstacles in a custom designed world, hence can perform safe testing of the TD3 deep reinforcement learning model and view outcomes of iterative development.
CloudCompare	Point Cloud map to Gazebo world file conversion	Enables conversion of real-world point cloud maps obtained from SLAM into lightweight Gazebo-compatible environments, facilitating accurate and realistic simulation environments for navigation testing. Can customize number of parameters to reduce model size for loading easily in Gazebo without crashing.

Visual Studio Code & Git	Code editor for Python connected to GitHub repository	Used for visualizing, testing and debugging Python code. VS Code's integration with Git was desirable as it streamlines versioning and ease of Git commits to my repository.
Meta SAM 2	Object segmentation and annotation	Based on a point or bounding box set by the user, instance segmentation of the selected object is performed and propagated through all frames in a video.
Jupyter Lab	Running Meta SAM 2 for interactive object annotation tasks	Meta SAM 2 is officially supported in Jupyter notebook (hosted on Google Colab), local installation environment to annotate objects for YOLO model training purposes.
Voila & IPYWidgets	Web-based user interface for object segmentation and autonomous annotation	Development of an easy-to-use web GUI from a Jupyter Notebook, providing a simple way to finetune and assess the quality of multiclass segmentation performed on input videos.
RealSense Viewer	Camera feed visualization	Allows visualization of real-time camera input, so can observe the perspective from the robot's height and view how adjustments affect YOLO performance.
Ultralytics YOLO (v8s and v9m models)	Object detection and recognition	Offers accurate detection of multiple objects within each frame with low latency, supports CUDA and GPU usage when training and testing onboard the robot.

3.2 Hardware Tools

Table 4. Hardware tools used in the project

Hardware Tool	Purpose	Description
Quadruped robot	Robot performing autonomous navigation and visualization	A multipurpose, all terrain and omnidirectional, quadruped robot, classified as a ground mobile vehicle. This is the core robot meant to carry out autonomous navigation with obstacle avoidance and object detection, supported with additional sensors.
Joystick Controller	Manual control of quadruped robot	This is used to control the movement of the quadruped robot especially when there is need to direct its movements in order to perform SLAM or override the autonomous capabilities.
LiDAR Sensor	Obstacle distance measurement	Transmits and receives laser beams in the surroundings and measures the distance to objects within its range thus providing real time LiDAR data that paves way for safe path planning using DRL.
RealSense Camera	Environmental perception	Provides both RGB imaging and depth imaging which enables the robot to have continuous visual awareness of the environment which is very important when it comes to identifying objects and avoiding obstacles while moving.

Nvidia Jetson Orin Nano Developer Kit	DRL algorithm and YOLO object detection training and testing	Provides a means for GPU assisted training and assessing the DRL algorithm and the YOLO object detection model on a PC before deploying on the robot for practical use.
Nvidia Jetson AGX Orin Developer Kit	Onboard deployment of algorithms on robot	Acts as the primary on-board computer for execution of the ROS services created for both DRL and object detection algorithms on the quadruped robot.
Battery Kit	Power supply for Nvidia Jetson AGX Orin	Provides power to the Nvidia Jetson AGX Orin on the robot to ensure that the unit is operational throughout autonomous exercises.

3.3 Work Breakdown Structure

This project starts with Research and Initial Planning. In this phase of the project, an overview of existing literature about autonomous navigation and object tracking is conducted so that a clear technical roadmap can be devised. As we go along, we focus the attention on the DRL algorithms with respect to the TD3 algorithm in particular because it has proven effective in indoor navigation applications. In addition to that, a roundup of You Only Look Once (YOLO) algorithms is done in order to find out the best one for use in real-time object detection.

At this point, the System Architecture Design phase is concerned with establishing the aims and the technical specifications in respect of the navigation and object detection systems. Issues of code style and compatibility are considered to enable the systems work seamlessly on given hardware such as the robot and Jetson Orin devices. A number of

software components are selected ready for integration with the system, which include ROS 2, the relevant version of YOLO and also SAM 2. The high-level software design is then drawn up showing integration of TD3 for navigation, and the combination of YOLO and SAM 2 for object detection and information processing.

The next phase is Simulation and Environment Setup in which virtual worlds are built using Gazebo for the testing of indoor navigation and training on DRL. For the initial basic environment, there are simulated walls that internally share a simple maze design that will be used as the training ground for the DRL navigation algorithms. To enhance the realism of the training environment, point cloud mapped images are converted to model files and used in the Gazebo environment.

The Algorithm Development phase involves modifications of TD3, YOLO, and SAM 2. The specific parameters of the TD3 algorithm are optimized for implementing indoor navigation, while altering the settings of the YOLOv9 model for custom object detection within the site including adjusting the configuration (YAML) file for more accuracy. An internal SAM 2 based graphical user interface (GUI) is designed to aid in annotation work, and the verified images and text files for training with YOLO are routed into appropriate folders using a data pipeline. Both the TD3 and YOLO algorithms are installed and operationalized in ROS 2 as services so that they can be used on demand.

For the Testing and Optimization phase, extensive unit testing is performed in both simulated and real environments. Several versions of TD3 are tested in terms of navigation performance and collision-avoidance agility for their accuracy and speed, in order to be analyzed. Hyperparameters are adjusted to get the maximum performance, and still the focus is on making the object detection inference times as short as possible, particularly on the Nvidia Jetson AGX Orin, in order to get faster results.

In the stage of Real-World Deployment, the adjusted navigation and detection algorithms are placed into the Jetson AGX Orin – the hardware used is based on the robot compatibility – and trials on the field are carried out. Latency, detection, and how well each algorithm works in practice, are tested in the field. Repair of the residual problems in the applications or underlying systems and their further testing is done, which guarantees the safe and effective deployment of the system.

The project comes to an end with Documentation and Presentation. It is expected to have a detailed account justifying the choice of algorithms. This account further illustrates the results of the project and discusses the stage of development it is possible to reach in the future. Other related materials like presentation slides and videos for in-house review with formal presentation of the project are done to highlight the project accomplishments and its impact.

3.4 Functional Requirements

3.4.1 Autonomous Navigation using Quadruped Robot

The robot is expected to move around the indoor area of size 30m x 30m without human intervention. This is done by employing the TD3 DRL agent so that dynamic path planning and dynamic path alterations are possible, which entails moving through an area without a map and where static and moving barriers are present without collisions. The task of the system is to change the course in real-time to avoid obstacles which is targeted at >80% navigation performance achieved in simulation and real-world scenarios. Furthermore, the obstacle avoidance system is realized in such a manner that it can guarantee the proximity threshold of 20 cm is maintained at minimum.

3.4.2 Real-Time Object Detection with YOLOv9 model

This model will recognize all objects present in the field of view. It aims at a detection precision of >90% in the working area while its detection delay is not more than 100 ms so as to allow effective and continuous display of the detected V objects.

3.4.3 Data Annotation using SAM 2

There is always a need for creating annotated datasets for YOLOv9 to learn to recognize different custom objects' classes from new videos. For effective in-house video object annotation, SAM 2 software is utilized which helps in marking objects through an easy user-based prompt window. It has a GUI that allows real-time user annotation as well as modification of object classes. The intended goal of the annotation process is to achieve >90% accuracy in annotating objects and cut down the amount of time taken to manually annotate objects by at least 80% from traditional approaches.

3.5 Non-functional Requirements

The system's dependability is one of the factors that is guaranteed so that the navigation or detection algorithms function as intended even when the lighting conditions and other hindrances change. In order for a system to be reliable, the target failure rate must be significantly low on all real-life applications.

Another aspect of the design of systems concerns their scalability where the system is capable of adding more object classes as well as accommodating larger indoor areas. The robustness of the algorithm includes the capability of enhanced data loads and prolonged hours of performance without deterioration of efficiency for the purpose of future growth without internal sabotaging of the system.

Toward usability, the goal is anchored on helping the operators interact with the system with ease which includes a simple graphical user interface offered by SAM 2 for the users involved in video labelling and data set formation. In addition, the system provides easy navigation controls that are expected with the joystick interface to enable the operation of quadruped robot.

In order to meet data privacy and security standards, object annotations as well as all data manipulation are performed inside the local environment only, thereby avoiding reliance on any cloud services. In addition, restrictions to access are strictly enforced for any data stored or processed within the system in a Jetson Orin board, in order to protect any sensitive data contained in the system.

With regard to portability, the algorithms and models are constructed to operate on different versions of ROS and are compatible with other Nvidia devices providing room for future use. System components can furthermore be transferred to other unmanned ground vehicle (UGV) system architectures with few modifications, facilitating flexible implementation.

Maintainability is assured through modular code, which make it easier to upgrade certain services in the program later without affecting other ones. Each module is separately provided with detailed instructions that help in maintenance of the system easily and also helps in ensuring that the system is sustained for a longer period.

Chapter 4

Methodology

4.1 Autonomous Navigation

4.1.1 Overview and System Architecture

The autonomous navigation system for this project was implemented in steps, each increasingly more advanced than the last. This design's evolution was leveraged towards the latest advances in artificial intelligence for robotics by utilizing deep reinforcement learning techniques, particularly the Twin Delayed Deep Deterministic Policy Gradients (TD3) algorithm. The TD3 algorithm has been observed to be very effective for problems that need stable navigation with continuous action space. The main focus was to master autonomous navigation which includes dynamic obstacle avoidance, path planning and execution, and decision making in real time, all of which take place in the most complex indoor workspace settings of a size 30x30 meters.

The realization of the system employed ROS 2 (Robot Operating System 2) as the medium of interaction between the elements and the system, where development took place in a simulated environment of the Gazebo in order to train and assess the performance of the navigation system. Thanks to the modular and distributed design of ROS 2, it was possible to create a scalable system, where the nodes could be developed, tested and modified independently. It also enabled complete testing and simulation, ensuring an effective system was developed prior to any deployment into the field.

4.1.2 Implementation of Reinforcement Learning

The structure of the TD3 algorithm is based on the Actor-Critic model, in which, two critics are employed to improve the Q-value estimation and hence the stability of the decisions made by the system is improved. The actor network takes states in and produces the corresponding actions out, whereas, the critic network evaluates the state-action pairs and enables actions in the system to be optimal. Each of the network components is classified into a layer containing each of its respective networks:

Actor Network:

Actors take in the present state as their input forwards it over several layers equipped with ReLU activations such that it can grasp non-linear complexities within the environment. The last layer possesses a tanh activation function capable of mapping the actions within a certain range of values which is normalized to the limitations of the robotic system being controlled.

Critics Networks (Q1 and Q2):

Two Q networks are utilized to predict their Q values to improve the learning process's persistence. Each critic is configured with two input groups: one for a given state, and the other for an action. The output of these branches is fused and passed through several more layers in order to generate a Q value. Because of the twin critic architecture, which uses two networks to estimate the value of a given state, overestimation bias common with single network structure is eliminated thus enhancing performance in decision-making.

In order to enhance the performance and robustness of the TD3 algorithm, a number of changes were made. To make the actions more exploratory, noise was introduced into the actions using an Ornstein-Uhlenbeck process. This is especially helpful in more complicated scenarios where the agent's task success relies on a great deal of exploration.

Gradient clipping with a maximum norm of 2.0 was used to avoid very large updates causing instability in the training process. Policy updates were deferred for the most part, and done every few iterations only (POLICY_UPDATE_FREQUENCY) so that more learning would be achieved in every iteration. The target networks were updated gradually to enhance flexibility in policy transfers and this promoted stability of learning during episodes.

4.1.3 Reward Function Optimization

A balanced reward function is important for the agent as it helps to navigate the agent's actions towards the desired behaviour. The project progressively improved the methods of motivating the agent, adding those which encouraged better navigation, overcoming obstacles and reaching the target purposefully. The final reward system in the series (version 29) integrated a number of functions in order to equalize reaching the target while keeping orientation and minimizing other objects:

Distance and Angle Penalty function: A penalty term based on the distance with respect to the goal and angle with respect to orientation. This ensures that the agent is always attempting to bring the distance to the goal down, albeit at the correct angle.

$$r_distance = - (abs(goal_dist*goal_angle)/10)$$

Obstacle Avoidance Reward: The reward structure comprised of layers among other systems that imposed penalties as the obstacles were neared encouraging careful movements and discouraging risky actions.

if min_obstacle_dist < 0.6:

$$r_obstacle = -10 + abs(action_angular*3.14)$$

elif min_obstacle_dist < 1.2:

$$r_{\text{obstacle}} = -2 + 2 * (\text{abs}(\text{action_angular}) - 2 * \text{abs}(\text{action_linear} * 2 - 1))$$

else:

$$r_{\text{obstacle}} = -2 + 2 * (\text{action_linear} - \text{abs}(\text{goal_angle} - (\text{action_angular} * 3.14)))$$

Linear Velocity Adjustment: This reward component modifies rewards given for linear velocity, depending on the position of the goal, in order to encourage optimal movement without losing the desired orientation.

if abs(goal_angle) < 0.2:

$$r_{\text{vlinear}} = -0.25 * (1 / \text{abs}(\text{action_linear} + 0.5))$$

Terminal rewards: As a motivation for performance or as a punishment for not accomplishing, terminal rewards were also introduced.

Goal Reached: +20000; Timeout: +10000; Tumble: -10000; Collision: -40000

4.1.4 Scaling of the Environment as well as Optimization

The system was subjected to progressive tests within environments of increasing complexity, each of which sought the navigation competence of the agent in a particular way.

Test 1: An initial 5x5 square static maze was used for basic tests involving the TD3 algorithm and its corresponding reward system.

Test 2: The addition of moving obstacles in this environment introduced the need for the agent to learn dynamic collision avoidance through the change of trajectory within the playing field. This can be viewed in Figure 1.

Test 3: A version of the system in which a larger and realistic virtual working environment was created from point cloud data while optimizing the mesh for performance in Gazebo. Adjustments were made on the level of model rendering and model size to avoid incurring a high graphic processing unit cost while retaining a high level of the environment detail. This can be viewed in Figure 2.

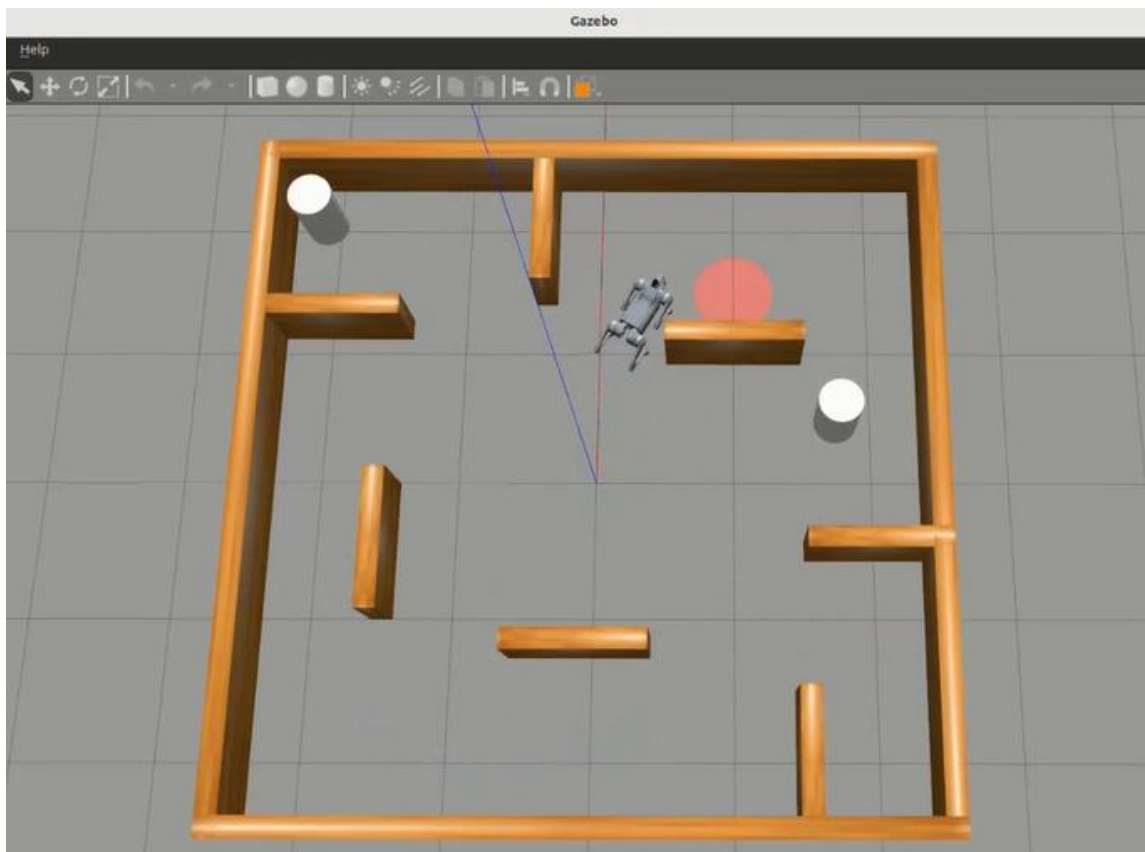


Figure 1. DRL simulation in a 5x5m maze-like environment. The red goal indicates the goal position and the white cylinders are dynamic obstacles that move around in the maze.

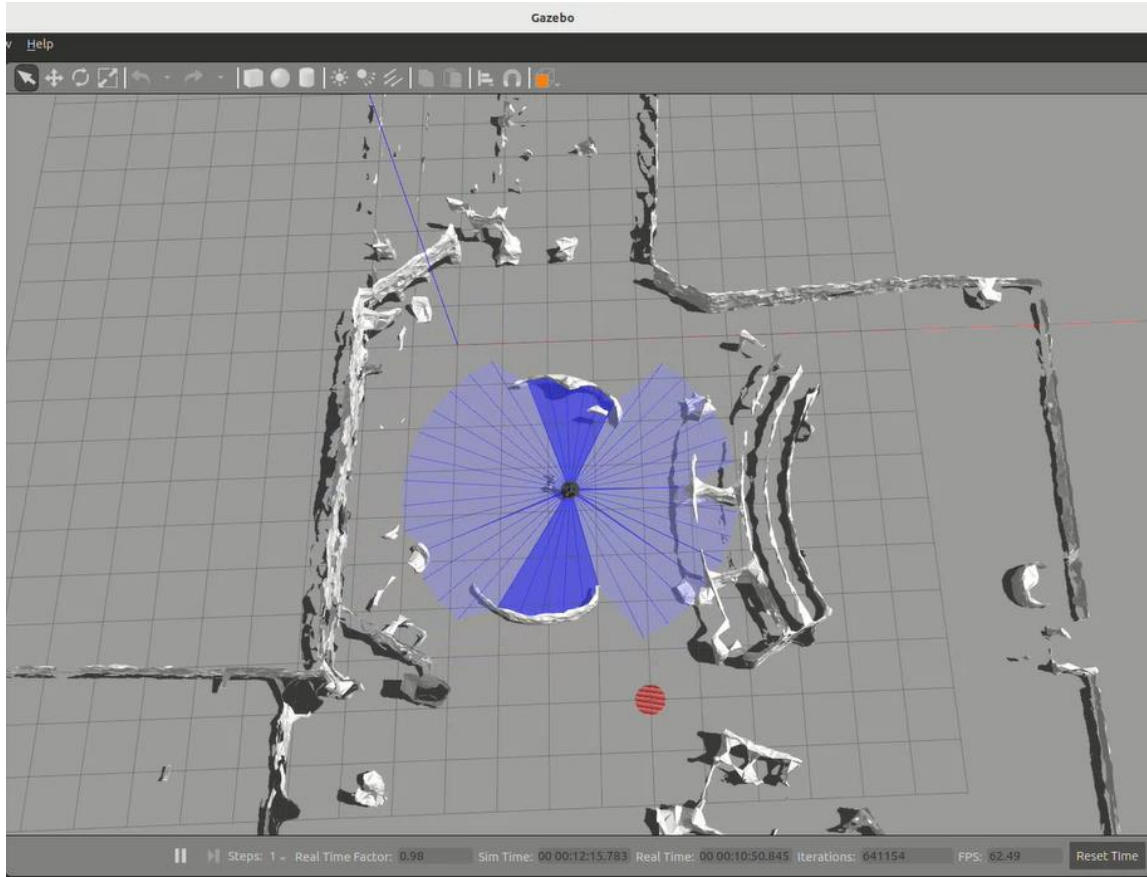


Figure 2. DRL simulation in a 30x30m workspace environment created from a Point Cloud map. The blue area around the robot represents the LiDAR scan.

4.1.5 ROS Integration and Creation of Nodes

The messaging and service architecture of ROS 2 formed the basis of the navigation system by providing a structured real-time communication network with different nodes, where processes were able to talk to each other and share data. This architecture was centered on several mainstream components, which consisted of several nodes, each assigned a specific task.

The TD3 Agent Node was responsible for taking in sensory information from the environment and producing commands to the robot in reaction to the environment. It was within this node that the TD3 of the agent was located, whose role was to analyze the different sensory information to determine how to navigate the robot. for instance,

directing the robot to change its direction or even stop to prevent colliding with an object. After every input, the TD3 agent node would then issue an action output in a form of a message to correspond to the next action of the robot.

The Environment Interface Node was in charge of the communication between ROS 2 and the Gazebo environment. This node was very significant as it allowed proper coordination of the data from the simulation with the actual state of the robot, hence allowing a better data transfer between the Gazebo environment and the TD3 model. It carried out activities that concerned the modification of the environment, like moving obstacles or creating extra obstacles in order to simulate motion.

In order to keep track of the various transformations occurring in the environment and the robot itself, the State Observer Node kept on evaluating and storing space state information, in real time, such as the location and orientation of the robot, as well as obstacles detected. This operator node functioned as the observer of the system by providing the TD3 agent with real time accurate state information which was essential for carrying out its purpose.

At last, the Action Publisher Node was in charge of implementing the proposed actions by the TD3 Agent Node by means of publishing commands for adjusting the speed of the robot. This node was the intermediary function between making decisions and taking actions, as it enabled issuing movement commands to the robot along the path planned by the TD3 agent.

In addition to the principal nodes, additional service implementations in ROS 2 were designed to address the requirements for effective and safe navigation: The State Service created a means to access the state of the robot and its surroundings at a given instant and this information was utilized by the TD3 agent in determining the appropriate trajectory

to navigate. The service enabled any node within the ROS 2 framework to query for and obtain current state information whenever needed, hence all actions were taken at the most recent state of the environment.

A Reset Service was created with the purpose of resetting the simulation framework before every simulation run. This capability made it possible to conduct the experimental trials in systematic manner by allowing the robot to start from specific locations in the clean environment which incubated regular training periods.

The Collision Avoidance System checks whether or not the vehicle was about to crash and reported crash events. This system was fundamental during all navigation processes since it was responsible for taking actions to avoid hit or register the hit for further investigation.

Topic organization in ROS 2 enabled deployment of the communication mechanisms for each component so that they publish and subscribe to relevant data. This further enhanced modularity and orderly flow of data:

The `/cmd_vel` topic was responsible for the robot's movement and transmitted the velocity commands, so the movement instructions could be sent from the Action Publisher Node directly to the robot.

Data about the robot's travel and turn on the `odom` topic along with the current location of the robot in space over time, which helped the TD3 agent understand how far and in what direction the robot is moving.

The `/scan` topic was dedicated to the ingested laser data of distances to obstacles and contributed the information to the TD3 Agent Node regarding distance to any obstacles in the agent's forward trajectory. This information helped the agent to avoid obstacles by changing the route.

The /state topic provided essential information regarding the state of the robot and environment to all other nodes when they required such information.

Finally, the purpose of the /reward topic was related to the signals presented to the TD3 agent in order to trigger some changes in its internal model. The agent was also provided with a negative feedback/warning signal for actions which were not aimed at the desired goal, such as, for example, successfully completing the route without colliding with any object.

4.1.6 Modifications in TD3 Algorithm

The first modification that was made is also termed as the Policy Update Frequency, which primarily looked at how often the policy network would be trained. Instead of having a constant update rate, that latency was varied with respect to the operating environment. In less cluttered regions, the updates could be less frequent, using resources effectively while ensuring that the system remains stable. However, in more complex areas, such as those with high exclusionary elements with moving parts, the update frequency was ramped up to ensure the algorithm can keep up with changes in the environment. Through this responsive modification, the chances of excessive training or overfitting were minimized while still allowing the model to be performant in different situations.

Also, a specialized Noise Implementation was implemented aimed to encourage exploratory actions in an appropriate manner. The TD3 agent was powered with a variant of the Ornstein-Uhlenbeck (OU) noise mechanism which was more appropriate in that it encouraged continuous rather than episodic exploration. While accomplishing tasks, the agent was provided with additional variability in actions through the use of OU noise which allowed for the exploration of new paths under new conditions which had not been available to the agent before. This was very ideal in large and complicated areas because

it allowed the agent to go beyond the range of the first set of actions which might have been small and look for better or safer options.

The Network Architecture was deliberately revised with the aim of improving the input data processing and comprehension capabilities of the algorithm. The hidden layers of the actor and critic networks were modified so as to increase their feature extraction capability, enabling the algorithm to identify and comprehend subtle details within an environment and as a result the decisions made would be better. In addition temp was employed so as to avoid any given update from making a drastic modification to the network weights even though annoying changes are possible with these types of policies which is even more critical with continuous action spaces. For additional stability improvements, the method included batch normalization which ensured that the inputs for the different layers were kept consistent and also controlled the output of the network from data range variations. This architecture optimization increased the learning integrity of the agent and made the learning process easier even for complicated navigation patterns.

In order to increase Training Stability even more, a number of advanced techniques were added to the training of the model. One of primary techniques was experience replay with prioritized sampling in order for the agent to pay more attention to valuable past experience, for example, overcoming obstacles or avoiding crashes. This attention-focused strategy enabled training efficiency for the algorithm trained in response to situations important for safety, allowing faster training for safety-critical systems. Also, even as the model ‘grows’, the learning rates were dynamic and changed.

4.1.7 Metrics for Evaluation

The performance of the agent was appraised at all times with certain metrics that were important in making successive changes. For instance, the measures such as success rate,

which is articulated in a percentage of seeking goals achieved, per episode return, which is average cumulative reward in a given episode, the rate of colliding with items while in motion, or path return which is the measure of how far or how possible the path taken is stood for effective and efficient navigation. In addition, hyperparameters such as the learning rate, discount factor and policy noise were suitably optimized in order to improve performance. Gradual variations in environmental complexity, for instance, rates of increase in obstacles and change in the starting position of the agent allowed the agent to operate and be efficient under different conditions, encouraging strong and dynamic navigation abilities.

4.1.8 Adjustments for Real World Testing and On-board Deployment

Core algorithm parameters formulated prior, include discount factor (γ) of 0.995 putting greater emphasis on future rewards with the learning rate at 0.005 to allow gradual progression of learning. A soft update rate (τ) of 0.005 was imposed to allow the target networks to be updated at a slow pace while policy noise was set at 0.2 as a function of clipping noise at ± 0.4 to mitigate the effects of action space exploration. The alternating parameter measurement takes place every two steps in order to reduce the variance. Concerning the network architecture, hidden layer size was fixed at 512 neurons, with two-dimensional action space which consisted of linear and angular velocity. The learning process was capable due to a batch size of 128, and a replay buffer, which contained 1,000,000 samples. Within parameters specific to the environment, linear and rotational maximum speeds were limited to 1.0 m/s and 1.5 rad/s – with 2.0 rad/s for the real robot, in addition to a 3.5 m reach with the LiDAR. Collision boundaries were set at 0.25 m for simulation and 0.11 m for real-world experiments, while, the threshold for reaching the goal was set to 0.3 m for simulation as well as 0.25 m for the robot.

There were also necessary alterations or enhancements made when transitioning from simulating the robot's operation to operating the robot indoors, as safety was the primary concern when performing any activities in an open space. Out of caution, an intervention control system based on key-press was used for interference in navigation when a robotic system is performing autonomously. This system uses a key-press subscription mechanism that allows movement only when a designated key is pressed. Movement commands were allowed or prohibited using a Boolean flag system. A zero-velocity fallback, wherein the robot does not move if the specific key is not pressed, was an additional safety precaution.

if key_pressed_flag:

 twist.linear.x = action_linear

 twist.angular.z = action_angular

else:

 twist.linear.x = 0.0

 twist.angular.z = 0.0

In this regard, the ROS 2 topic structure has been designed for the control of motion and sensor input to operate in real time. Some of the important topics were:

/key_press (Bool) – press to allow for movement,

/cmd_vel (Twist) – supplying the robot with velocity commands,

/scan (LaserScan) – data from the robot's LiDAR and

/odom (Odometry) – location and orientation of the robot.

Practical considerations regarding the implementation of all technologies necessitated adaptations in sensor data processing, movement settings and safety measures. In particular, a LiDAR correction factor of 0.40 m was added to improve sensor processing due to the inaccuracy in the placement of the LiDAR sensor. Also, the sample count for the LiDAR scan was limited to 40 to enhance the speed of processing and all distance readings were limited to 3.5 m.

The robot's physical parameters, for instance, the maximum limits of collision and goal tolerance zones, and positioning accuracy and operational stability zones had a lot to do with the settings in movement parameters. Maximum velocities were also adjusted to obtain stable movement of the quadruped, taking into consideration the weight and balance limitations typical of real applications.

4.2 Autonomous Annotation

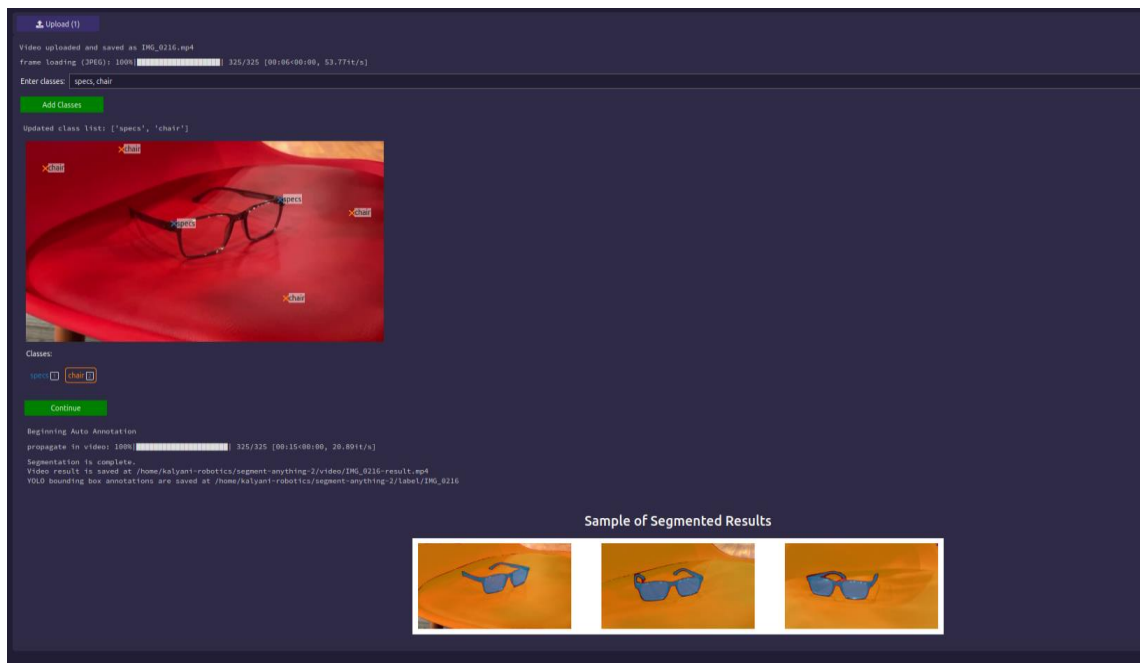


Figure 3. Web application for autonomous annotation using Voila in Jupyter Lab.

A custom autonomous annotation system was developed leveraging Meta's Segment Anything Model 2 (SAM 2) to facilitate efficient video segmentation and annotation. The system includes a graphical user interface (GUI) that enables users to upload videos, define regions of interest using positive and negative prompts, and automatically generate YOLO-format bounding box annotations for each frame.

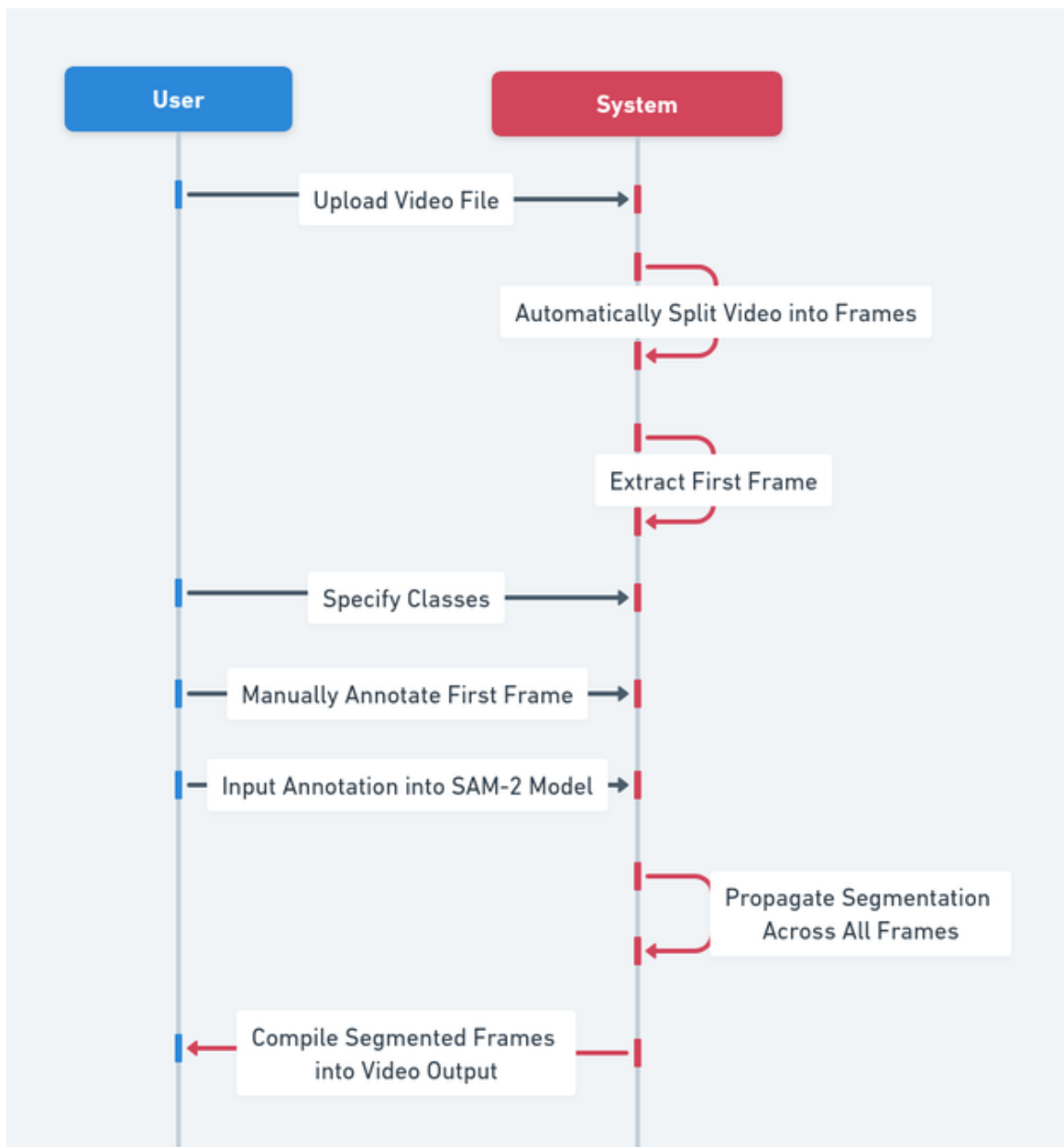


Figure 4. Flowchart of sequence of operations in the annotation application.

The system is optimized for usage in environments with CUDA installed and applying GPU computing power when present and competent while also providing a fallback for traditional central processing units. Certain optimizations were carried out for NVIDIA devices with compute capability equal or greater than 8.0 allowing for TensorFloat-32 precision in CUDA matrix multiplications and DNN operations.

The application is flexible as in users can import videos in several formats such as .mp4, .mov, and .avi. Before any automation of the presentation of the video is undertaken, it is still feasible to process the video and frame by frame extract pictures out of it. It is envisaged that the frames will be sampled at 0.5 magnification for a balance between speed of processing and details captured. The resulting images will be referred to in the subsequent sections as frames, and particular naming conventions with five digits padded by zeroes will be adopted when converting and storing the frames into .jpeg images.

To enable users to draw areas of interest, a BBoxWidget was developed. Users have the option to add as many object classes as they like using a text box and separating the class names with commas. The system also permits positive and negative area marking to enhance accuracy in segmentation. Each of the object classes has a specific number and a color from the available color combinations.

Specifically, 'sam2_hiera_large' has been used for the system in question. The system remains in inference-mode tracking across the frames to keep track of the object(s) being segmented. Prompts for the model are provided in the form of click points defined by the user: positive on the areas to inform them that they should be segmented in the area, negative to the ones that segmentation masks shouldn't be drawn in.

The system propagates segmentation masks across video frames and preserves object tracking by using unique object identifiers. In case of binary segmentation mask logits are

generated at a threshold of 0. The architecture allows tracking multiple simultaneous objects in the scene.

Annotations are created in YOLO bounding box format which is composed of class ID and center x and y coordinates normalized, as well as the width and height normalized. Segmentation masks are transformed into bounding boxes using mask-to-xyxy conversion. Annotation for every frame is kept in separate text file with its respective frame index.

The entire management of output data follows a specific and logical structure as can be seen from the arrangement of the directories. Three individual folders are set aside for the raw frames, annotated frames and the yolo annotations. Naming conventions are standardized throughout these folders to allow for ease of access through Python coding.

If for instance a new object would move into the video halfway through its length, the user can opt to begin from a different frame rather than the first one by default in an effort to re-use the available segmentations with the new annotation so as to overwrite the existing output file in an efficient manner.

In addition, the system contains a fixed class list of 50 entities frequently found in the office for example, couches, work pods, electronics and office supplies. Such classification can be supplemented with or changed in accordance with the need. Moreover, the interface makes it possible to introduce classes on the fly at the time of annotation, making it possible to turn the application to the user's needs even more.

4.3 Autonomous Visualization

The self-governing video segmentation mechanism not only provides a way of annotation which is quick but also assists in automating the process of creating a dataset for training a model. The system automatically generates folders for raw frames, annotated frames,

and YOLO annotation text files. These annotations are then shuffled and set aside in a 2:1:1 ratio for training, validation, and testing of the different YOLO models so as to avoid overfitting models.

At first, the implemented system trained on the small model of YOLOv8. With a low amount of annotated data (approximately 300 annotated images) in the initial phases of manual annotation prior to the application development, the inference time remained high and the accuracy of the model remained low at around 60%.

This situation changed dramatically regarding both the quality and quantity of the data available, after recording multiple videos, and the use of the annotation system which assisted in expanding the dataset to more than 10,000 annotated images. With a larger dataset, the training was more effective. Moreover, by making the system compatible with the on-board GPU and CUDA, the inference time reduced significantly from 6 seconds to 25 milliseconds. Besides speeding up the processing, such improvement also enhanced the performance of the model where the accuracy rose from 60% to 99%. The transition to the GPU enhanced setup allowed a shift from YOLOv8s to YOLOv9m, along with the correct software configuration regarding TensorFlow and TensorVision package versions.

For vocalization, OpenAI was used for text to speech with a queue of 25 frames noting the occurrence of bounding boxes being detected with class names. Then, only if the bounding box count was 10 or greater will the spoken text string include the object name. This was done to ensure that accidental detections or false positives were not spoken aloud, so only if consistent detection of the object is occurring is the model confident enough to declare it as a detected object aloud. Since the camera stream is continuous, two sets were designed wherein objects, once detected, would be transferred to another set so that repeated class names are not said aloud, and only new objects that appear in the field of view are.

Chapter 5

Results and Discussion

For the autonomous navigation TD3 algorithm, graph visualizations were created for keeping track of counts of successes, collisions, tumbles, and timeouts across the episodes during training, as well as actor loss, critic loss, and average reward across 10 episodes.

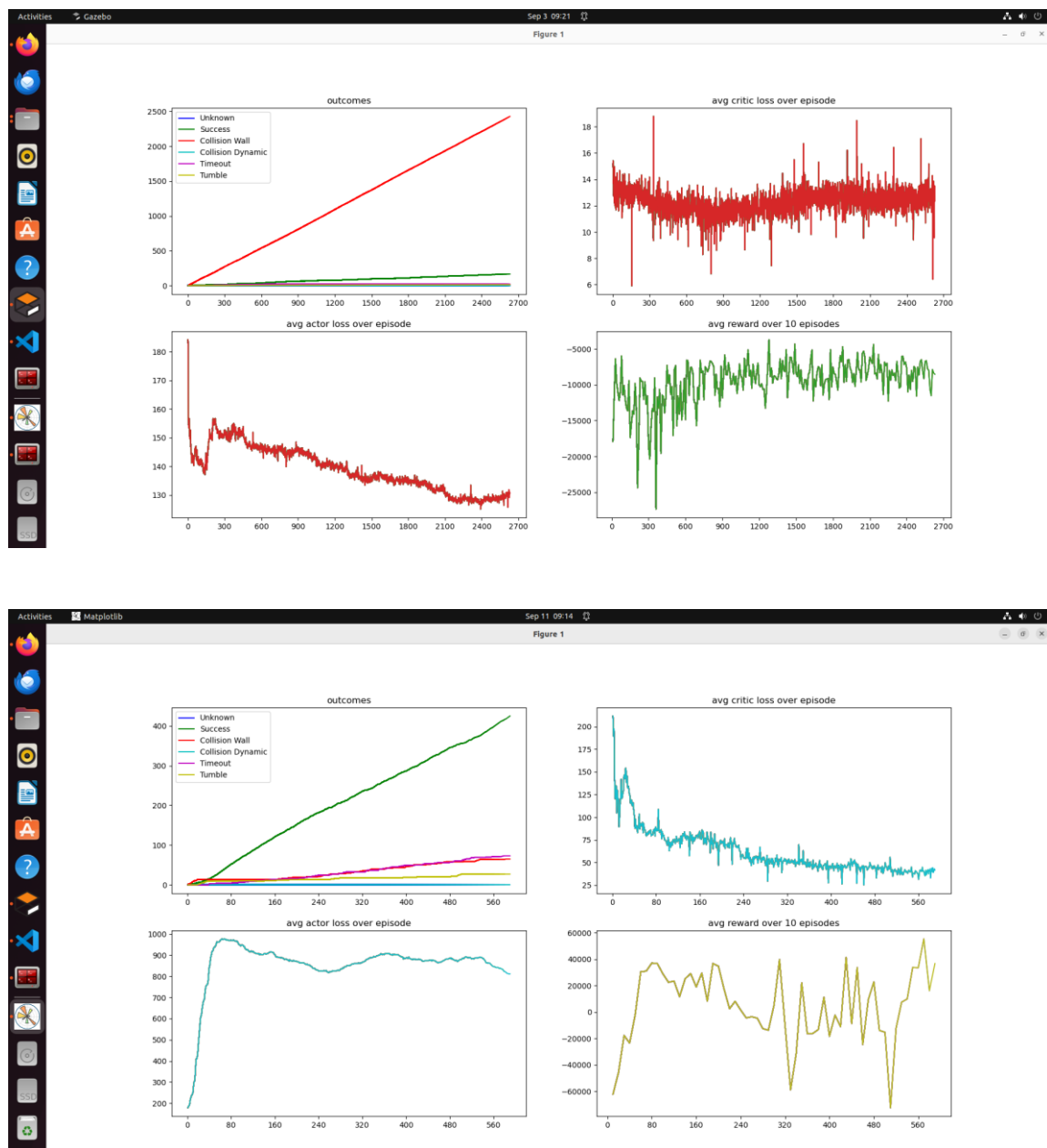


Figure 5. a) Results obtained from standard TD3 algorithm when deployed onto 30x30m environment. **b)** My optimized TD3 algorithm designed for large environments.

As can be seen in Figure 5, most research papers and standard algorithms for deep reinforcement learning only consider smaller environments with nearby goals and few or no obstacles. The graph in a) was derived from the same algorithm which showed commendable performance in the 5x5m maze environment, because for longer distances and further goals, there is much need for improvisation of the hyperparameters and reward function. The results obtained in b) were after a total of 29 trials wherein attempts to modify reward function design were made with gradual increasing of successes. The reason for the high actor loss is purely due to the reason that the environment being much larger demands more time for exploration which means many more states being explored by the agent and thus loss increases during this period. Simulations in the environment with this version showed optimal performance of obstacle avoidance, hence this version was selected for on-board deployment.

The annotation application was further improved based on edge case testing and new requirements, with the implementation of negative areas for segmentation.



Figure 6. Example of negative segmentation masks in spectacles annotation.

The application also supports segmentation of similar class names with automatic numbering attached for differentiation. Then by filtering the numbers at the end of the class names detected, it becomes possible to save separate bounding box co-ordinates with the same class id, desirable for optimum YOLO training.

YOLOv8s was initially trained with the manually annotated dataset, and its precision-confidence graph can be seen in Figure 7. The collated dataset from all videos were used for training YOLOv9m and the performance metrics have been visualized in Figures 9, 10, and 11. The visualizations are aligned with the practical observation that the model performs to an excellent degree and is able to correctly identify nearly 60 custom objects with ideal differentiation of positions and lower false positives and false negatives. YOLOv10 was avoided at the time due to quite a few open issues, but with v11 recently announced, it is possible to shift towards v10 training to observe if performance in terms of accuracy and latency can be improved.

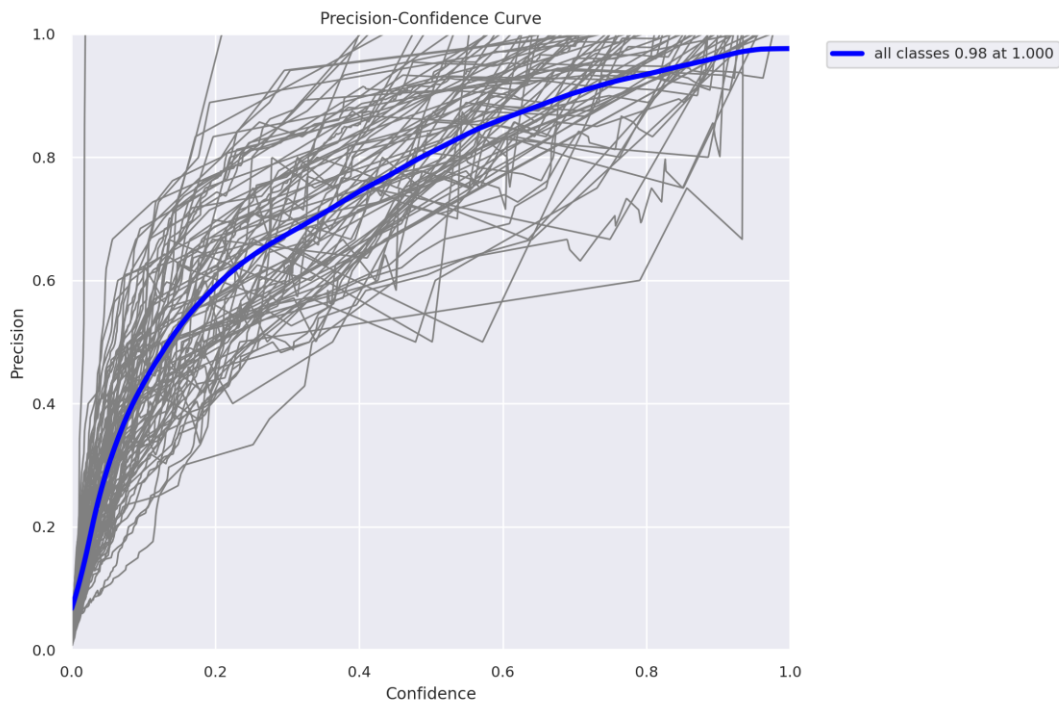


Figure 7. Precision-confidence curve for YOLOv8s on dataset of 300 images.

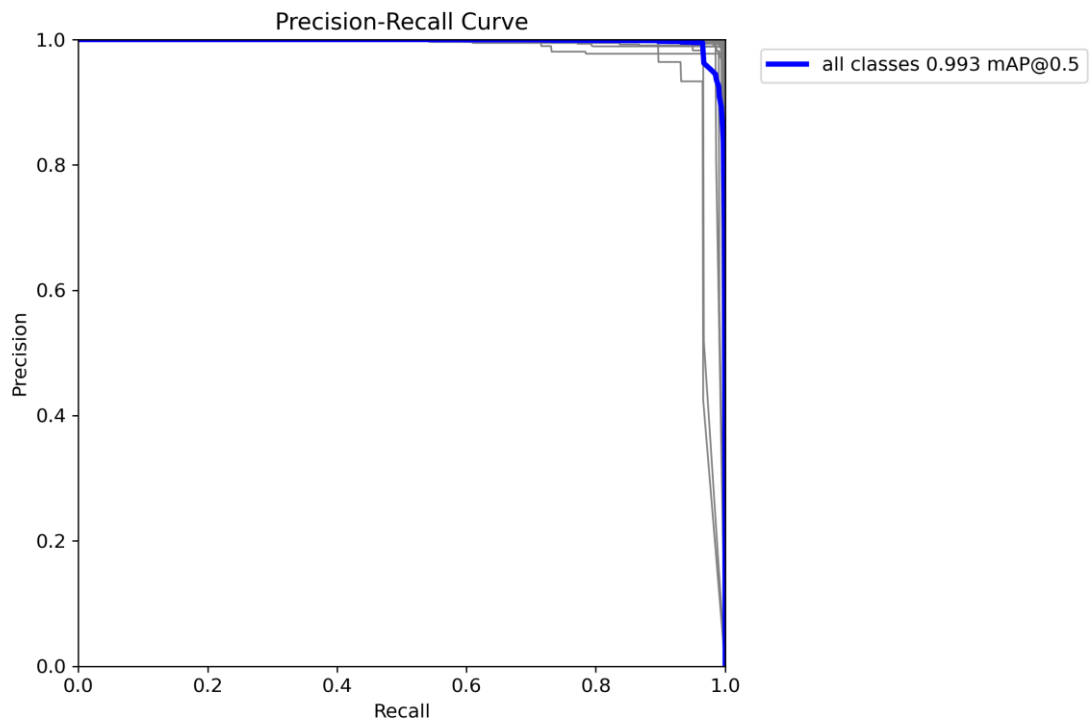


Figure 8. Precision-recall curve for YOLOv9m on dataset of over 10000 images.

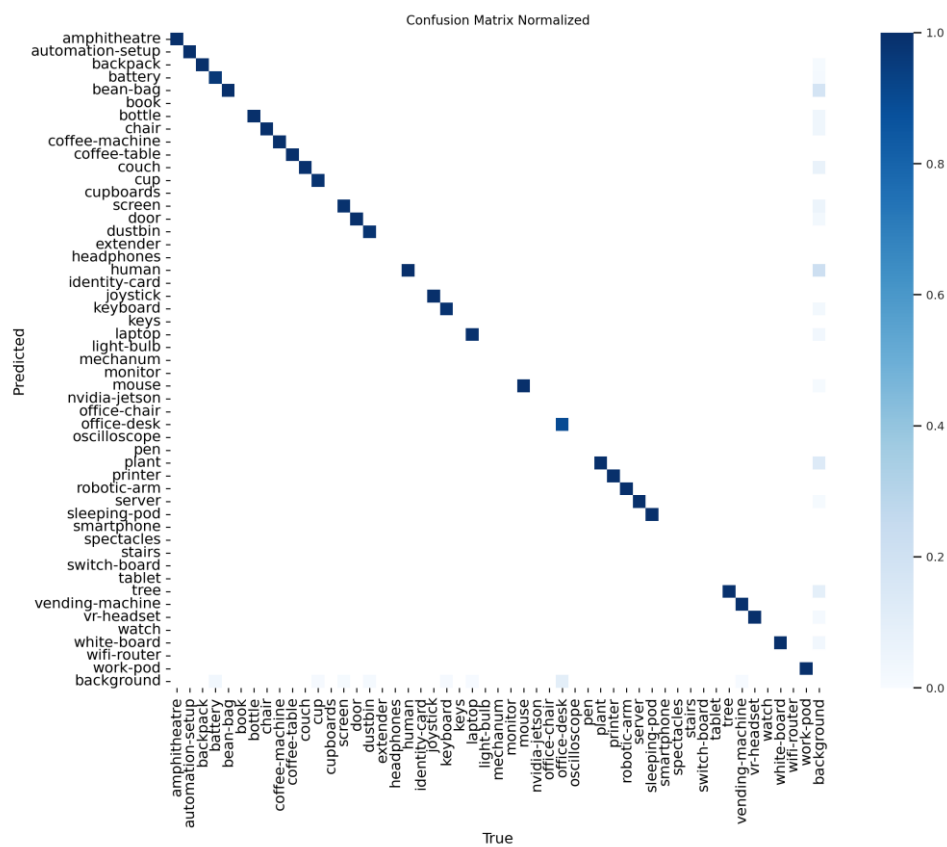


Figure 9. Normalized confusion matrix for YOLOv9m.

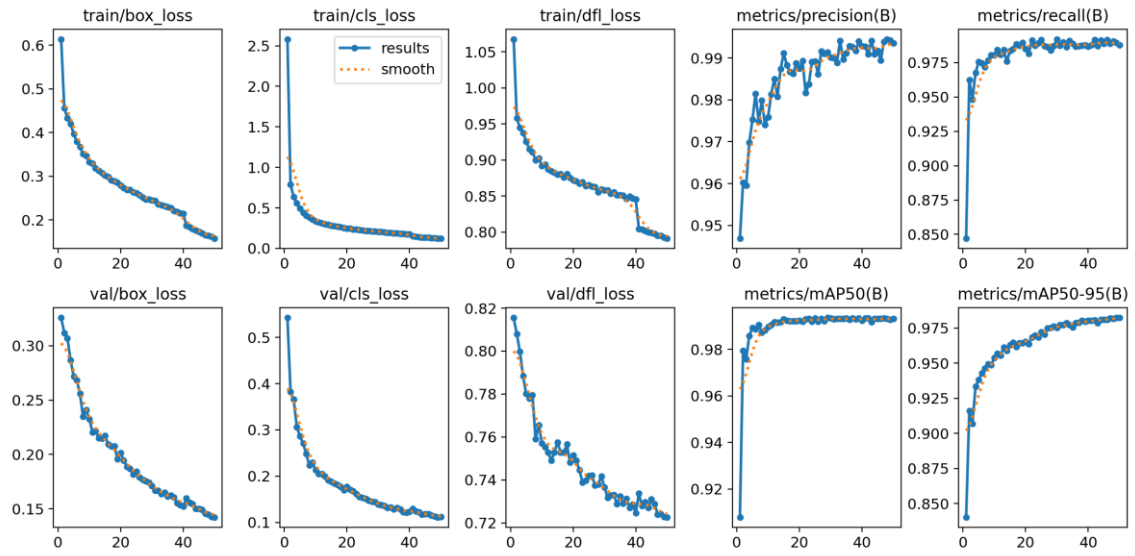


Figure 10. Graphs for loss, precision, and recall for training and validation across 50 epochs for YOLOv9m.

Chapter 6

Conclusion and Future Scope

5.1 Conclusion

The aim of this project was to design, build, and demonstrate advanced algorithms for autonomous navigation and object detection within indoor environments, and indeed the project was accomplished successfully with the culmination of several approaches, combining Deep Reinforcement Learning with the YOLO object detection framework and advanced annotation tools based on SAM 2. Improvements in Twin Delayed Deep Deterministic Policy Gradient (TD3) reinforcement learning algorithm enable the quadruped robot to maneuver efficiently within a cluttered 30 x 30m office-like environment, without the need of storing a map. The adjustments made to the navigation model not only help achieve the project objectives but also afford high flexibility and performance in different indoor environments beyond the one for which it was designed. The employment of YOLOv9 for object detection in the project, as well as a custom locally deployed annotation tool based on SAM 2, made it possible to create and manage a dataset quickly and accurately while protecting sensitive information. This work has laid a strong possibility for the implementation of embedded artificial intelligence systems in Unmanned Ground Vehicles (UGVs) thus enhancing the scope for autonomous navigation and visualization in practical situations.

5.2 Future Scope

This initiative, as the initial stage of the precepts of more generalized and adaptable artificial intelligence systems, opens a number of possibilities on how to improve both algorithm parameters and system features in general. Further developments might imply

the use of YOLOv11 and further versions in order to enhance detection accuracy and reduction of latency, as well as possible new versions of SAM to be released that support multi-object tracking and internal segmentation in fast moving and cluttered scenarios. In terms of navigation, it may be possible to improve arithmetic efficiency of active responses in complicated situations by integrating path planning techniques with the TD3-oriented deep reinforcement learning (DRL) approach, in relation to speed and dependability in dynamic environments. Extended research of this kind also takes the work a step closer to the aspirations of the new field of Artificial General Intelligence (AGI), as it creates building blocks for artificial intelligence systems which are capable of learning and reasoning in different environments. Looking ahead, enhancing these core designs and intensifying their implementation will facilitate the further development of AGI focused UGVs that will be able to work continuously, learn and change as well as make high-level decisions in real-life complex settings. In this way, the project becomes a very important first step towards creating autonomous agents endowed with human level intelligence, who can independently but in a complex manner interact with their environment.

References

- [1] M. Quigley *et al.*, “ROS: An open-source Robot Operating System,” IEEE International Conference on Robotics and Automation, 2009. Available: <http://robotics.stanford.edu/~ang/papers/icraoss09-ROS.pdf>
- [2] G. Brockman *et al.*, “OpenAI Gym,” *arXiv (Cornell University)*, Jan. 2016, doi: 10.48550/arxiv.1606.01540
- [3] L. Tai and M. Liu, “Towards Cognitive Exploration through Deep Reinforcement Learning for Mobile Robots,” *arXiv (Cornell University)*, Jan. 2016, doi: 10.48550/arxiv.1610.01733.
- [4] L. Tai and M. Liu, “A robot exploration strategy based on Q-learning network,” *2016 IEEE International Conference on Real-time Computing and Robotics (RCAR)*, Angkor Wat, Cambodia, 2016, pp. 57-62, doi: 10.1109/RCAR.2016.7784001.
- [5] L. Tai, G. Paolo, and M. Li, “Virtual-to-real Deep Reinforcement Learning: Continuous Control of Mobile Robots for Mapless Navigation,” *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Mar. 2017, doi: 10.48550/arxiv.1703.00420.
- [6] M. Pfeiffer, M. Schaeuble, J. Nieto, R. Siegwart and C. Cadena, “From perception to decision: A data-driven approach to end-to-end motion planning for autonomous ground robots,” *2017 IEEE International Conference on Robotics and Automation (ICRA)*, Singapore, 2017, pp. 1527-1533, doi: 10.1109/ICRA.2017.7989182.

- [7] A. Faust *et al.*, “PRM-RL: Long-range Robotic Navigation Tasks by Combining Reinforcement Learning and Sampling-based Planning,” *2018 IEEE International Conference on Robotics and Automation (ICRA)*, Brisbane, Australia, 21–25 May 2018, pp. 5113–5120, doi: 10.48550/arxiv.1710.03937.
- [8] A. Francis *et al.*, “Long-Range Indoor Navigation with PRM-RL,” *IEEE Trans. Robot.*, pp. 1115–1134, 2020, doi: 10.48550/arxiv.1902.09458.
- [9] H.-T. L. Chiang, J. Hsu, M. Fiser, L. Tapia, and A. Faust, “RL-RRT: Kinodynamic Motion Planning via Learning Reachability Estimators From RL Policies,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4298–4305, Oct. 2019, doi: 10.1109/lra.2019.2931199.
- [10] J.-J. Zeng, L. Qin, Y. Hu, Q. Yin, and C. Hu, “Integrating a Path Planner and an Adaptive Motion Controller for Navigation in Dynamic Environments,” *Applied Sciences*, vol. 9, no. 7, pp. 1384–1384, Apr. 2019, doi: 10.3390/app9071384.
- [11] A. Iyer and A. Mahadevan, “Collision Avoidance Robotics Via Meta-Learning (CARML),” *arXiv (Cornell University)*, Jul. 2020, doi: 10.48550/arxiv.2007.08616.
- [12] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel, “Reverse Curriculum Generation for Reinforcement Learning,” *2017 Conference on Robot Learning (CoRL)*, 2017, doi: 10.48550/arXiv.1707.05300.
- [13] A. Stooke and P. Abbeel, “Accelerated Methods for Deep Reinforcement Learning,” *arXiv (Cornell University)*, Jan. 2018, doi: 10.48550/arxiv.1803.02811.

- [14] S. Fujimoto, H. Hoof, and D. Meger, “Addressing Function Approximation Error in Actor-Critic Methods,” *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, pp. 1587-1596, doi: 10.48550/arXiv.1802.09477
- [15] Y. Yin, Z. Chen, G. Liu, and J. Guo, “A Mapless Local Path Planning Approach Using Deep Reinforcement Learning Framework,” *Sensors*, vol. 23, no. 4, p. 2036, Feb. 2023, doi: 10.3390/s23042036.
- [16] Y. Cui, H. Zhang, Y. Wang and R. Xiong, “Learning World Transition Model for Socially Aware Robot Navigation,” *2021 IEEE International Conference on Robotics and Automation (ICRA)*, Xi'an, China, 2021, pp. 9262-9268, doi: 10.1109/ICRA48506.2021.9561973.
- [17] S. Wen, Y. Zhao, X. Yuan, Z. Wang, D. Zhang, and L. Manfredi, “Path planning for active SLAM based on deep reinforcement learning under unknown environments,” *Intelligent Service Robotics*, vol. 13, no. 2, pp. 263–272, Jan. 2020, doi: 10.1007/s11370-019-00310-w.
- [18] J. Gao, W. Ye, J. Guo, and Z. Li, “Deep Reinforcement Learning for Indoor Mobile Robot Path Planning,” *Sensors*, vol. 20, no. 19, p. 5493, Sep. 2020, doi: 10.3390/s20195493.
- [19] R. Cimurs, J. H. Lee, and I. H. Suh, “Goal-Oriented Obstacle Avoidance with Deep Reinforcement Learning in Continuous Action Space,” *Electronics*, vol. 9, no. 3, p. 411, Feb. 2020, doi: 10.3390/electronics9030411.
- [20] T. P. Lillicrap *et al.*, “Continuous control with deep reinforcement learning,” *arXiv (Cornell University)*, Jul. 05, 2020. doi: 10.48550/arXiv.1509.02971

- [21] O. Canal and F. Taschin, “Self-learned vehicle control using PPO,” Feb. 11, 2020.
[Online]. Available: https://campusai.github.io/pdf/autonomous_driving.pdf
- [22] P. Mirowski *et al.*, “Learning to Navigate in Complex Environments,” *arXiv (Cornell University)*, 2016, doi: 10.48550/arxiv.1611.03673.
- [23] H. Shi, L. Shi, M. Xu, and K.-S. Hwang, “End-to-End Navigation Strategy With Deep Reinforcement Learning for Mobile Robots,” *IEEE Transactions on Industrial Informatics*, vol. 16, no. 4, pp. 2393–2402, Apr. 2020, doi: 10.1109/TII.2019.2936167.
- [24] Y. Zhu *et al.*, “Target-driven visual navigation in indoor scenes using deep reinforcement learning,” *2017 IEEE International Conference on Robotics and Automation (ICRA)*, Singapore, 2017, pp. 3357–3364, doi: 10.1109/ICRA.2017.7989381.
- [25] G. Brunner, O. Richter, Y. Wang, and R. Wattenhofer, “Teaching a Machine to Read Maps With Deep Reinforcement Learning,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018, doi: 10.1609/aaai.v32i1.11645.
- [26] U. Gawande, K. Hajari, and Y. Golhar, “SIRA: Scale illumination rotation affine invariant mask R-CNN for pedestrian detection,” *Applied Intelligence*, Jan. 2022, doi: 10.1007/s10489-021-03073-z.
- [27] C. Bouras and E. Michos, “An online real-time face recognition system for police purposes,” *2022 International Conference on Information Networking (ICOIN)*, Jeju-si, Korea, Republic of, 2022, pp. 62–67, doi: 10.1109/ICOIN53446.2022.9687212.

- [28] Z. Guo, W. Liao, Y. Xiao, P. Veelaert, and W. Philips, “Weak segmentation supervised deep neural networks for pedestrian detection,” *Pattern Recognition*, vol. 119, p. 108063, Nov. 2021, doi: 10.1016/j.patcog.2021.108063.
- [29] Y. Jamtsho, P. Riyamongkol, and R. Waranusast, “Real-time license plate detection for non-helmeted motorcyclist using YOLO,” *ICT Express*, 2021, doi: 10.1016/j.icte.2020.07.008.
- [30] J. Fu, C. Zhao, Y. Xia, and W. Liu, “Vehicle and wheel detection: a novel SSD-based approach and associated large-scale benchmark dataset,” *Multimedia Tools and Applications*, vol. 79, no. 17–18, pp. 12615–12634, Jan. 2020, doi: 10.1007/s11042-019-08523-y.
- [31] C. Li *et al.*, “A parallel down-up fusion network for salient object detection in optical remote sensing images,” *Neurocomputing*, vol. 415, pp. 411–420, Nov. 2020, doi: 10.1016/j.neucom.2020.05.108.
- [32] R. Mehta and C. Ozturk, “Object detection at 200 Frames Per Second,” *arXiv (Cornell University)*, 2019, doi: 10.48550/arxiv.1805.06361.
- [33] J. Zhou, Y. Tian, W. Li, R. Wang, and Z. Luan, “LADet: A Light-weight and Adaptive Network for Multi-scale Object Detection,” *Proceedings of Machine Learning Research*, vol. 101, pp. 912–923, 2019, [Online]. Available: <https://proceedings.mlr.press/v101/zhou19a/zhou19a.pdf>
- [34] Z. Zou and Z. Shi, “Random Access Memories: A New Paradigm for Target Detection in High Resolution Aerial Remote Sensing Images,” vol. 27, no. 3, pp. 1100–1111, Mar. 2018, doi: 10.1109/tip.2017.2773199.

- [35] W. Wang, J. Shen, R. Yang, and Fatih Porikli, “Saliency-Aware Video Object Segmentation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 1, pp. 20–33, Jan. 2018, doi: 10.1109/tpami.2017.2662005.
- [36] K. Behrendt, L. Novak, and R. Botros, “A deep learning approach to traffic lights: Detection, tracking, and classification,” *2017 IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, doi: icra.2017.7989163.
- [37] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017, doi: 10.1109/tpami.2016.2577031.
- [38] A. Kirillov *et al.*, “Segment Anything,” *arXiv (Cornell University)*, Apr. 2023, doi: 10.48550/arxiv.2304.02643.
- [39] C. Zhang *et al.*, “A Comprehensive Survey on Segment Anything Model for Vision and Beyond,” *arXiv (Cornell University)*, May 2023, doi: 10.48550/arxiv.2305.08196.
- [40] J. Zhu, Y. Qi, and J. Wu, “Medical SAM 2: Segment medical images as video via Segment Anything Model 2,” *arXiv (Cornell University)*, Aug. 2024, doi: 10.48550/arXiv.2408.00874.
- [41] J. Ma *et al.*, “Segment Anything in Medical Images and Videos: Benchmark and Deployment,” *arXiv (Cornell University)*, Aug. 2024, doi: 10.48550/arxiv.2408.03322.

- [42] D. Li *et al.*, “FusionSAM: Latent Space driven Segment Anything Model for Multimodal Fusion and Segmentation,” *arXiv (Cornell University)*, Aug. 2024, doi: 10.48550/arxiv.2408.13980.
- [43] N. Ravi *et al.*, “SAM 2: Segment Anything in Images and Videos,” *arXiv (Cornell University)*, Aug. 2024, doi: 10.48550/arxiv.2408.00714.
- [44] C. Zhang *et al.*, “Segment Anything for Videos: A Systematic Survey,” *arXiv (Cornell University)*, 2024, [Online]. Available: <https://arxiv.org/pdf/2408.08315>
- [45] Y. Zhang and Z. Shen, “Unleashing the Potential of SAM2 for Biomedical Images and Videos: A Survey,” *arXiv (Cornell University)*, Aug. 2024, doi: 10.48550/arxiv.2408.12889.
- [46] J. Yu *et al.*, “SAM 2 in Robotic Surgery: An Empirical Evaluation for Robustness and Generalization in Surgical Video Segmentation,” *arXiv (Cornell University)*, Aug. 2024, doi: 10.48550/arxiv.2408.04593.